

Mata Kuliah Praktikum Unggulan Universitas Gunadarma : Kecerdasan Buatan dengan Nvidia DGX A100

tugas membuat modif data dari batu gunting kertas menggunakan sistem ai markov

batu gunting kertas bot lawan bot

1)bot pertama menggunakan random angka

2)bot kedua menggunakan markov chain base (AI)

```
In [1]: #botbattle!!!! The algorithm based bot is Bot 1 and the Markov Chain Based one i
# Inisialisasi variabel untuk menyimpan pilihan bot dan skor
import random

bot1choice = 0
bot2choice = 0
bot1prevchoice = 0
bot2prevchoice = 0
results = ""
results2 = ""
bot1score = [0,0,0]
bot2score = [0,0,0]

# Matriks transisi untuk menyimpan frekuensi kemunculan pasangan pilihan (rock,
# Digunakan untuk menghitung probabilitas Markov Chain
buildTMatrix = {'rr': 1, 'rp': 1, 'rs': 1, 'pr': 1, 'pp': 1, 'ps': 1, 'sr': 1, '
buildTMatrixL = {'rr': 1, 'rp': 1, 'rs': 1, 'pr': 1, 'pp': 1, 'ps': 1, 'sr': 1,
buildTMatrixT = {'rr': 1, 'rp': 1, 'rs': 1, 'pr': 1, 'pp': 1, 'ps': 1, 'sr': 1,

n = 3
m = 3
# Matriks probabilitas untuk kondisi Menang, Kalah, Seri
tMatrix = [[0] * m for i in range(n)]
tMatrixL = [[0] * m for i in range(n)]
tMatrixT = [[0] * m for i in range(n)]

# Probabilitas awal masing-masing pilihan (Rock, Paper, Scissors)
probabilitiesRPS = [1/3,1/3,1/3]

def init():
    # Menginisialisasi permainan bot
    global bot1prevchoice
    global bot1choice
    global bot2prevchoice
    global bot2choice
    global results
    global results2

    # Asumsi inisial: pilihan sebelumnya dan saat ini adalah rock (0)
```

```

bot1prevchoice, bot1choice = bot1(0,0,"Tied!") #inital assuming previous cho
bot2prevchoice, bot2choice = bot2(0,0,"Tied!") #inital assuming previous cho
results = checkWin(bot1choice,bot2choice)
if (results == "Win!"):
    results2 = "Lose!"
elif (results == "Lose!"):
    results2 = "Win!"
else:
    results2 = "Tied!"

# Memulai pertarungan sebanyak 100.000 ronde
fight(100000)

def fight(rounds):
    # Fungsi utama yang menjalankan iterasi ronde permainan bot
    global bot1prevchoice
    global bot1choice
    global bot2prevchoice
    global bot2choice
    global results
    global results2
    global bot1score
    global bot2score
    choose = ["Rock","Paper","Scissors"]
    for i in range(0,rounds):
        # Asumsi inisial: pilihan sebelumnya dan saat ini adalah rock (0)
        bot1prevchoice, bot1choice = bot1(bot2prevchoice,bot2choice,results2)
        #print ("Bot 1 chose %s" % choose[bot1choice])
        # Bot 2 menentukan pilihan menggunakan Markov Chain
        bot2prevchoice, bot2choice = bot2(bot1prevchoice,bot1choice,results)
        #print ("Bot 2 chose %s" % choose[bot2choice])
        #print (buildTMatrix)
        #print (buildTMatrixL)
        #print (buildTMatrixT)
        # Mengecek siapa yang menang ronde ini
        results = checkWin(bot1choice,bot2choice)
        if (results == "Win!"):
            results2 = "Lose!"
        elif (results == "Lose!"):
            results2 = "Win!"
        else:
            results2 = "Tied!"
        #print (results)
        if (results == "Win!"):
            # Memperbarui skor masing-masing bot
            bot2score[1] += 1
            bot1score[0] += 1
        elif (results == "Lose!"):
            bot1score[1] += 1
            bot2score[0] += 1
        else:
            bot2score[2] += 1
            bot1score[2] += 1
    # Mencetak hasil akhir
    print ("Bot1 won %s times, lost %s times, and tied %s times.\n\nTo check:\n\

def bot1(prev,choit,res):
    # Bot 1: Menggunakan algoritma berbasis aturan (rule-based)
    # Bergantung pada 'streak' kemunculan dan riwayat kemenangan
    choices = ["Rock", "Paper", "Scissors"]

```

```

prevChoice = prev
choice = choit
prevMachineChoice = ""
result = res
streak = 0
won = 0
alt = 0
numoff = 0

if (prevChoice == choice):
    streak += 1
else:
    streak -= 1
    if (streak < 0):
        streak = 0
# Pola untuk mengalahkan pilihan lawan sebelumnya
winprev = prevChoice + 1
if (winprev > 2):
    winprev = 0
if (choice == winprev):
    alt += 1
else:
    alt -= 1
    if (alt < 0):
        alt = 0
# Menentukan pilihan selanjutnya berdasarkan dominasi pola Lawan
if (alt > 3):
    machineChoice = winprev + 1
    if (machineChoice > 2):
        machineChoice = 0
elif (streak > 3):
    machineChoice = prevChoice - 2
    if (machineChoice < 0):
        machineChoice += 3
elif (won > 9):
    machineChoice = random.randint(0, 2)
elif (won > 3 and won < 10):
    machineChoice = prevChoice
else:
    if (result == "Win!"):
        machineChoice = prevChoice - 2
        if (machineChoice < 0):
            machineChoice += 3
    elif (result == "Lose!"):
        machineChoice = prevChoice + 1
        if (machineChoice > 2):
            machineChoice -= 3
        machineChoice -= 2
        if (machineChoice < 0):
            machineChoice += 3
    else:
        machineChoice = random.randint(0, 2)

result = checkWin(choice, machineChoice)

if (result == "Win!"):
    won += 1
else:
    won -= 2
    if (won < 0):

```

```

        won = 0
    return choit, machineChoice

def checkWin(user, machine):
    # Mengecek kondisi menang/kalah/seri (0: Rock, 1: Paper, 2: Scissors)
    win = False
    tie = False
    if (user == 0):
        if (machine == 2):
            win = True
            tie = False
        elif (machine == 1):
            win = False
            tie = False
        elif (user == 0):
            tie = True
        else:
            print ("Something wierd happened and machine was: %s" % machine)
    elif (user == 1):
        if (machine == 0):
            win = True
            tie = False
        elif (machine == 2):
            win = False
            tie = False
        elif (machine == 1):
            tie = True
        else:
            print ("Something wierd happened and machine was: %s" % machine)
    else:
        if (machine == 1):
            win = True
            tie = False
        elif (machine == 0):
            win = False
            tie = False
        elif (machine == 2):
            tie = True
        else:
            print ("Something wierd happened and machine was: %s" % machine)

    if (tie == True):
        return "Tied!"
    elif (win):
        return "Win!"
    else:
        return "Lose!"

def bot2(previ,choit,res):
    # Bot 2: Menggunakan Markov Chain
    # Memilih aksi selanjutnya berdasarkan probabilitas kejadian historis
    global probabilitiesRPS
    choices = ["Rock","Paper","Scissors"]
    choi = ['r','p','s']
    prevChoice = previ
    probRock = 0
    probPaper = 0
    probScissors = 0
    result = res

```

```

choice = choit
# Membangun matriks transisi terbaru untuk menebak pilihan lawan selanjutnya
transMatrix = buildTransitionProbabilities(prevChoice, choice, result)
machineChoice = random.randint(1, 100)
# Membaca probabilitas (Rock, Paper, Scissors)
probabilitiesRPS[0] = transMatrix[prevChoice][0]
probabilitiesRPS[1] = transMatrix[prevChoice][1]
probabilitiesRPS[2] = transMatrix[prevChoice][2]
# Menentukan range untuk pemilihan acak menggunakan bobot Markov
rangeR = probabilitiesRPS[0] * 100
rangeP = probabilitiesRPS[1] * 100 + rangeR
if (machineChoice <= rangeR):
    machineChoice = 1
elif (machineChoice <= rangeP):
    machineChoice = 2
else:
    machineChoice = 0
return choit, machineChoice

def buildTransitionProbabilities(pC, c, winloss):
    # Memperbarui frekuensi transisi state (kemunculan pilihan pasangan)
    global buildTMatrix
    global buildTMatrixL
    global buildTMatrixT
    choi = ['r', 'p', 's']

    # Update state history berdasarkan status Menang/Kalah/Seri
    if winloss == "Win!":
        for i, x in buildTMatrix.items():
            if ('%s%s' % (choi[pC], choi[c])) == i:
                buildTMatrix['%s%s' % (choi[pC], choi[c])] += 1
    elif winloss == "Tied!":
        for i, x in buildTMatrixT.items():
            if ('%s%s' % (choi[pC], choi[c])) == i:
                buildTMatrixT['%s%s' % (choi[pC], choi[c])] += 1
    else:
        for i, x in buildTMatrixL.items():
            if ('%s%s' % (choi[pC], choi[c])) == i:
                buildTMatrixL['%s%s' % (choi[pC], choi[c])] += 1

    return buildTransitionMatrix(winloss)

def buildTransitionMatrix(winlosstwo):
    # Membangun dan menormalkan Matriks Probabilitas Transisi Markov Chain
    global tMatrix
    global tMatrixL
    global tMatrixT

    if winlosstwo == "Win!":
        rock = buildTMatrix['rr'] + buildTMatrix['rs'] + buildTMatrix['rp']
        paper = buildTMatrix['pr'] + buildTMatrix['ps'] + buildTMatrix['pp']
        scissors = buildTMatrix['sr'] + buildTMatrix['ss'] + buildTMatrix['sp']
        choi = ['r', 'p', 's']
        for row_index, row in enumerate(tMatrix):
            for col_index, item in enumerate(row):
                a = int(buildTMatrix['%s%s' % (choi[row_index], choi[col_index])])
                if (row_index == 0):
                    c = a/rock

```

```

        elif (row_index == 1):
            c = a/paper
        else:
            c = a/scissors
        row[col_index] = float(c)
    return (tMatrix)
elif winlosstwo == "Tied!":
    rock = buildTMatrixT['rr'] + buildTMatrixT['rs'] +buildTMatrixT['rp']
    paper = buildTMatrixT['pr'] + buildTMatrixT['ps'] +buildTMatrixT['pp']
    scissors = buildTMatrixT['sr'] + buildTMatrixT['ss'] +buildTMatrixT['sp']
    choi = ['r','p','s']
    for row_index, row in enumerate(tMatrixT):
        for col_index, item in enumerate(row):
            a = int(buildTMatrixT['%s%s' % (choi[row_index],choi[col_index])])
            if (row_index == 0):
                c = a/rock
            elif (row_index == 1):
                c = a/paper
            else:
                c = a/scissors
            row[col_index] = float(c)
    return (tMatrixT)

else:
    rock = buildTMatrixL['rr'] + buildTMatrixL['rs'] +buildTMatrixL['rp']
    paper = buildTMatrixL['pr'] + buildTMatrixL['ps'] +buildTMatrixL['pp']
    scissors = buildTMatrixL['sr'] + buildTMatrixL['ss'] +buildTMatrixL['sp']
    choi = ['r','p','s']
    for row_index, row in enumerate(tMatrixL):
        for col_index, item in enumerate(row):
            a = int(buildTMatrixL['%s%s' % (choi[row_index],choi[col_index])])
            if (row_index == 0):
                c = a/rock
            elif (row_index == 1):
                c = a/paper
            else:
                c = a/scissors
            row[col_index] = float(c)
    return (tMatrixL)

init()

```

Bot1 won 523 times, lost 98894 times, and tied 583 times.

To check:

Bot2 won 98894 times, lost 523 times, and tied 583 times.

pada program batu gunting kertas ini akan menggunakan sistem markov chain

In [2]: `#!/usr/bin/env python3`

```

import random
# Inisialisasi variabel statistik pemain (Menang, Kalah, Seri)
winEas,loseEas,tieEas = 0.0,0.0,0.0

# Kamus frekuensi state markov (sebelumnya -> sekarang) untuk menang, kalah, dan
buildTMatrix = {'rr': 1, 'rp': 1, 'rs': 1, 'pr': 1, 'pp': 1, 'ps': 1, 'sr': 1, '
buildTMatrixL = {'rr': 1, 'rp': 1, 'rs': 1, 'pr': 1, 'pp': 1, 'ps': 1, 'sr': 1,
buildTMatrixT = {'rr': 1, 'rp': 1, 'rs': 1, 'pr': 1, 'pp': 1, 'ps': 1, 'sr': 1,

```

```

n = 3
m = 3
# Matriks probabilitas 3x3 untuk Markov Chain
tMatrix = [[0] * m for i in range(n)]
tMatrixL = [[0] * m for i in range(n)]
tMatrixT = [[0] * m for i in range(n)]

probabilitiesRPS = [1/3,1/3,1/3]

def markov():
    # Fungsi permainan utama untuk manusia melawan AI berbasis Markov
    global probabilitiesRPS
    choices = ["Rock", "Paper", "Scissors"]
    choi = ['r', 'p', 's']
    continuePlaying = True
    prevChoice = ""
    choice = 3
    probRock = 0
    probPaper = 0
    probScissors = 0

    # Meminta input awal pemain
    try:
        choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))
    except ValueError:
        print("you must enter an integer \n")

    if(choice > 2 or choice < 0):
        print("You must enter an integer less than three and greater than 0 \n")
        while(choice > 2 or choice < 0):
            print("You must enter an integer less than three and greater than 0 \n")
    # Meminta input awal pemain
    try:
        choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))
    except ValueError:
        print("you must enter an integer \n")

    # Pilihan mesin yang pertama kali adalah acak
    machineChoice = random.randint(0, 2)
    # Evaluasi hasil dan perbarui state pilihan sebelumnya
    result = checkWin(choice, machineChoice, 1)
    print("You chose %s" % choices[choice])
    print("The machine chose %s" % choices[machineChoice])
    print("You %s" % result)

    prevChoice = choice

    # Loop permainan berkelanjutan
    while(continuePlaying):
        choice = 3
        # Meminta input awal pemain
        try:
            choice = int(input("0: Rock, 1: Paper, 2: Scissors, 5: exit \n"))
        except ValueError:
            print("you must enter an integer \n")

        if((choice > 2 or choice < 0) and choice != 5):
            print("You must enter an integer less than three and greater than 0 or ch
            while((choice > 2 or choice < 0) and choice != 5):

```

```

        print ("You must enter an integer less than three and greater than 0 or
# Meminta input awal pemain
        try:
            choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))
        except ValueError:
            print("you must enter an integer \n")
# Pemain keluar, mencetak statistik permainan
if (choice == 5):
    print ("Thanks for Playing!")
    print ("You won %d times!" % int(winEas))
    print ("You lost %d times!" % int(loseEas))
    print ("You tied %d times!" % int(tieEas))
    percentWon = "{percent:.2%}".format(percent=(winEas / (winEas+loseEas+tieE
    print ("Your win percentage is %s from a total of %d games" % (percentWon,
    continuePlaying = False
else:
    # Membangun matriks dari aksi historis untuk prediksi selanjutnya
    transMatrix = buildTransitionProbabilities(prevChoice,choice,result)
    machineChoice = random.randint(1, 100)
    # Menyesuaikan probabilitas AI untuk melawan kecenderungan tindakan penggu
    probabilitiesRPS[0] = transMatrix[prevChoice][0]
    probabilitiesRPS[1] = transMatrix[prevChoice][1]
    probabilitiesRPS[2] = transMatrix[prevChoice][2]
    # Bobot pembagian kemungkinan AI memilih rock, paper, atau scissors
    rangeR = probabilitiesRPS[0] * 100
    rangeP = probabilitiesRPS[1] * 100 + rangeR
    if (machineChoice <= rangeR):
        machineChoice = 1
    elif (machineChoice <= rangeP):
        machineChoice = 2
    else:
        machineChoice = 0

    # Evaluasi hasil dan perbarui state pilihan sebelumnya
    result = checkWin(choice,machineChoice,1)
    prevChoice = choice
    print ("You chose %s" % choices[choice])
    print ("The machine chose %s" % choices[machineChoice])
    print("You %s" % result)

# Menampilkan Matriks Transisi Akhir
print("Your winning transition matrix is:\nr: %s\np: %s\ns: %s\n" % (tMatrix[0
print("Your losing transition matrix is:\nr: %s\np: %s\ns: %s\n" % (tMatrixL[0
print("Your tying transition matrix is:\nr: %s\np: %s\ns: %s\n" % (tMatrixT[0]

def buildTransitionProbabilities(pC,c,winloss):
    # Menambahkan kemunculan ke matriks yang merekam transisi state berdasarkan st
    global buildTMatrix
    global buildTMatrixL
    global buildTMatrixT
    choi = ['r','p','s']

    if winloss == "Win!":
        for i, x in buildTMatrix.items():
            if ('%s%s' % (choi[pC],choi[c]) == i):
                buildTMatrix['%s%s' % (choi[pC], choi[c])] += 1
    elif winloss == "Tied!":
        for i, x in buildTMatrixT.items():
            if ('%s%s' % (choi[pC],choi[c]) == i):
                buildTMatrixT['%s%s' % (choi[pC], choi[c])] += 1

```



```

else:
    for i, x in buildTMatrixL.items():
        if ('%s%s' % (choi[pC],choi[c]) == i):
            buildTMatrixL['%s%s' % (choi[pC], choi[c])] += 1

    return buildTransitionMatrix(winloss)

def buildTransitionMatrix(winlosstwo):
    # Menormalkan kemunculan matriks untuk probabilitas persentase desimal
    global tMatrix
    global tMatrixL
    global tMatrixT

    if winlosstwo == "Win!":
        rock = buildTMatrix['rr'] + buildTMatrix['rs'] +buildTMatrix['rp']
        paper = buildTMatrix['pr'] + buildTMatrix['ps'] +buildTMatrix['pp']
        scissors = buildTMatrix['sr'] + buildTMatrix['ss'] +buildTMatrix['sp']
        choi = ['r','p','s']
        for row_index, row in enumerate(tMatrix):
            for col_index, item in enumerate(row):
                a = int(buildTMatrix['%s%s' % (choi[row_index],choi[col_index])])
                if (row_index == 0):
                    c = a/rock
                elif (row_index == 1):
                    c = a/paper
                else:
                    c = a/scissors
                row[col_index] = float(c)
            return (tMatrix)
    elif winlosstwo == "Tied!":
        rock = buildTMatrixT['rr'] + buildTMatrixT['rs'] +buildTMatrixT['rp']
        paper = buildTMatrixT['pr'] + buildTMatrixT['ps'] +buildTMatrixT['pp']
        scissors = buildTMatrixT['sr'] + buildTMatrixT['ss'] +buildTMatrixT['sp']
        choi = ['r','p','s']
        for row_index, row in enumerate(tMatrixT):
            for col_index, item in enumerate(row):
                a = int(buildTMatrixT['%s%s' % (choi[row_index],choi[col_index])])
                if (row_index == 0):
                    c = a/rock
                elif (row_index == 1):
                    c = a/paper
                else:
                    c = a/scissors
                row[col_index] = float(c)
            return (tMatrixT)
    else:
        rock = buildTMatrixL['rr'] + buildTMatrixL['rs'] +buildTMatrixL['rp']
        paper = buildTMatrixL['pr'] + buildTMatrixL['ps'] +buildTMatrixL['pp']
        scissors = buildTMatrixL['sr'] + buildTMatrixL['ss'] +buildTMatrixL['sp']
        choi = ['r','p','s']
        for row_index, row in enumerate(tMatrixL):
            for col_index, item in enumerate(row):
                a = int(buildTMatrixL['%s%s' % (choi[row_index],choi[col_index])])
                if (row_index == 0):
                    c = a/rock
                elif (row_index == 1):
                    c = a/paper
                else:
                    c = a/scissors

```

```

        row[col_index] = float(c)
    return (tMatrixL)

def checkWin(user, machine, mode):
    # Logika evaluasi status menang/kalah
    win = False
    tie = False
    if (user == 0):
        if (machine == 2):
            win = True
            tie = False
        elif (machine == 1):
            win = False
            tie = False
        elif (user == 0):
            tie = True
        else:
            print ("Something wierd happened and machine was: %s" % machine)
    elif (user == 1):
        if (machine == 0):
            win = True
            tie = False
        elif (machine == 2):
            win = False
            tie = False
        elif (machine == 1):
            tie = True
        else:
            print ("Something wierd happened and machine was: %s" % machine)
    else:
        if (machine == 1):
            win = True
            tie = False
        elif (machine == 0):
            win = False
            tie = False
        elif (machine == 2):
            tie = True
        else:
            print ("Something wierd happened and machine was: %s" % machine)

    if (tie == True):
        checkStats(2, mode)
        return "Tied!"
    elif (win):
        checkStats(0, mode)
        return "Win!"
    else:
        checkStats(1, mode)
        return "Lose!"

def checkStats(wlt, modeChosen):
    # Mencatat statistik hasil kemenangan pengguna
    global winEas
    global loseEas
    global tieEas

    if (modeChosen == 1):
        if (wlt == 0):
            winEas += 1

```

```

elif (wlt == 1):
    loseEas += 1
else:
    tieEas += 1

markov()

```

```

You chose Rock
The machine chose Rock
You Tied!
You chose Paper
The machine chose Paper
You Tied!
You chose Scissors
The machine chose Scissors
You Tied!
Thanks for Playing!
You won 0 times!
You lost 0 times!
You tied 3 times!
Your win percentage is 0.00% from a total of 3 games
Your winning transition matrix is:
r: [0, 0, 0]
p: [0, 0, 0]
s: [0, 0, 0]

Your losing transition matrix is:
r: [0, 0, 0]
p: [0, 0, 0]
s: [0, 0, 0]

Your tying transition matrix is:
r: [0.25, 0.5, 0.25]
p: [0.25, 0.25, 0.5]
s: [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]

```

pada batu gunting kertas yang ketiga membuat pilihan di mana jika memilih easy maka akan menggunakan random jika memilih Intermediate maka akan menggunakan markov chain atau sisitem AI

```

In [3]: #!/usr/bin/env python3

import random
# Variabel styling terminal
X = '\033[0m'
Bold = '\033[1;36m'
HighB = '\033[1;44m'

# Variabel statistik skor untuk setiap mode permainan
winEas = loseEas = tieEas = winInt = loseInt = tieInt = winHard = loseHard = tie

hiddenfound = False

# Matriks Transisi Markov Chain untuk bot di tingkat kesulitan tinggi
buildTMatrix = {'rr': 1, 'rp': 1, 'rs': 1, 'pr': 1, 'pp': 1, 'ps': 1, 'sr': 1, '
buildTMatrixL = {'rr': 1, 'rp': 1, 'rs': 1, 'pr': 1, 'pp': 1, 'ps': 1, 'sr': 1,
buildTMatrixT = {'rr': 1, 'rp': 1, 'rs': 1, 'pr': 1, 'pp': 1, 'ps': 1, 'sr': 1,

```

```

n = 3
m = 3
tMatrix = [[0] * m for i in range(n)]
tMatrixL = [[0] * m for i in range(n)]
tMatrixT = [[0] * m for i in range(n)]

probabilitiesRPS = [1/3,1/3,1/3]

buildTMatrixrpsclsp = {'rr': 1, 'rp': 1, 'rsc': 1, 'rl': 1, 'rsp': 1, 'pr': 1, '
buildTMatrixlrpsclsp = {'rr': 1, 'rp': 1, 'rsc': 1, 'rl': 1, 'rsp': 1, 'pr': 1,
buildTMatrixtrpsclsp = {'rr': 1, 'rp': 1, 'rsc': 1, 'rl': 1, 'rsp': 1, 'pr': 1,

sheldon = 5
cooper = 5
tMatrixrpsclsp = [[0] * sheldon for i in range(cooper)]
tMatrixlrpsclsp = [[0] * sheldon for i in range(cooper)]
tMatrixtrpsclsp = [[0] * sheldon for i in range(cooper)]

probabilitiesrpsclsp = [1/5,1/5,1/5,1/5,1/5,1/5]
intro = """
Welcome to Rock Paper Scissors the Ultimate Version! There are four modes: Begin
Oh, and if you try to cheat off my code I WILL FIND YOU. Have fun!
P.S. There is a hidden mode that is pretty easy to figure out if you look at the
"""

print(Bold)
print(intro)
print(X)

def chooseMode():
    # Fungsi untuk memilih mode permainan dengan 4 tingkat kesulitan reguler dan 1
    mode = 6
    try:
        mode = int(input("What Mode do you want to play in? 1: beginner, 2: intermed
    except ValueError:
        print("you must enter an integer \n")

    if(mode > 4 and mode != 73):
        print ("You must enter an integer less than five \n")
        while(mode > 4 and mode != 73):
            try:
                mode = int(input("What Mode do you want to play in? 1: begginer, 2: int
            except ValueError:
                print("you must enter an integer \n")
        return mode

def easyMode():
    # Easy mode - Bot memilih secara sepenuhnya acak menggunakan random
    choices = ["Rock","Paper","Scissors"]
    continuePlaying = True
    continueGame = ""
    try:
        choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))
    except ValueError:
        print("you must enter an integer \n")

    if(choice > 2 or choice < 0):
        print ("You must enter an integer less than three and greater than 0 \n")
        while(choice > 2 or choice < 0):
            print ("You must enter an integer less than three and greater than 0 \n")
            try:

```

```

        choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))
    except ValueError:
        print("you must enter an integer \n")

    # Pilihan acak mesin
    machineChoice = random.randint(0, 2)
    result = checkWin(choice, machineChoice, 1)
    print ("You chose %s" % choices[choice])
    print ("The machine chose %s" % choices[machineChoice])
    print("You %s" % result)

while(continuePlaying):
    try:
        choice = int(input("0: Rock, 1: Paper, 2: Scissors, 5: exit \n"))
    except ValueError:
        print("you must enter an integer \n")

    if((choice > 2 or choice < 0) and choice != 5):
        print ("You must enter an integer less than three and greater than or equal to zero")
    while((choice > 2 or choice < 0) and choice != 5):
        print ("You must enter an integer less than three and greater than or equal to zero")
    try:
        choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))
    except ValueError:
        print("you must enter an integer \n")
    if (choice == 5):
        print ("Thanks for Playing!")
        continuePlaying = False
    else:
        # Pilihan acak mesin
        machineChoice = random.randint(0, 2)
        result = checkWin(choice, machineChoice, 1)

        print ("You chose %s" % choices[choice])
        print ("The machine chose %s" % choices[machineChoice])
        print("You %s" % result)

    #continueGame = input("Do you want to continue? Yes or No. \n")
    #while(continueGameCheck(continueGame) == "Wrong input"):
    # continueGame = input("Do you want to continue? Yes or No. This time act carefully")

    #if (continueGame == "Yes"):
    #    continuePlaying = True
    #else:
    #    continuePlaying = False

def intermediateMode():
    # Intermediate mode - Bot memilih kombinasi algoritma sederhana untuk melawan manusia
    choices = ["Rock", "Paper", "Scissors"]
    continuePlaying = True
    continueGame = ""
    prevChoice = ""
    prevMachineChoice = ""
    result = ""
    streak = 0
    won = 0
    alt = 0
    numoff = 0
    choice = 3

```

```

try:
    choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))
except ValueError:
    print("you must enter an integer \n")

if (choice > 2 or choice < 0):
    print(
        "You must enter an integer less than three and greater than
    while (choice > 2 or choice < 0):
        try:
            choice = int(input("0: Rock, 1: Paper, 2: Scisso
        except ValueError:
            print("you must enter an integer \n")
# Pilihan acak mesin
machineChoice = random.randint(0, 2)
result = checkWin(choice, machineChoice, 2)
if (result == "Win!"):
    won += 1
else:
    numoff += 1
    if (numoff == 3):
        won -= 3
        numoff = 0
    if (won < 0):
        won = 0

print("You chose %s" % choices[choice])
print("The machine chose %s" % choices[machineChoice])
print("You %s" % result)

prevChoice = choice
prevMachineChoice = machineChoice
streak += 1

while (continuePlaying):
    try:
        choice = int(input("0: Rock, 1: Paper, 2: Scissors, 5: e
    except ValueError:
        print("you must enter an integer \n")

    if ((choice > 2 or choice < 0) and choice != 5):
        print(
            "You must enter an integer less than three and great
        )
        while ((choice > 2 or choice < 0) and choice != 5):
            try:
                print(
                    "You must enter an integer less than
                )
                choice = int(input("0: Rock, 1: Paper, 2
            except ValueError:
                print("you must enter an integer \n")

    if (choice == 5):
        print("Thanks for Playing!")
        continuePlaying = False
    else:
        if (prevChoice == choice):
            streak += 1
        else:

```

```

        streak -= 1
        if (streak < 0):
            streak = 0
# Mesin merespon pola streak pengguna dan hasil pertandingan sebelumnya
        if (streak > 3):
            machineChoice = prevChoice - 2
            if (machineChoice < 0):
                machineChoice += 3
        elif (won > 9):
            print(
                "Yo. Stop cheating and keep your eyes off my
            )

# Pilihan acak mesin
        machineChoice = random.randint(0, 2)
        elif (won > 3 and won < 10):
            machineChoice = prevChoice
        else:
            if (result == "Win!"):
                machineChoice = prevChoice - 2
                if (machineChoice < 0):
                    machineChoice += 3
            elif (result == "Lose!"):
                machineChoice = prevChoice + 1
                if (machineChoice > 2):
                    machineChoice -= 3
                machineChoice -= 2
                if (machineChoice < 0):
                    machineChoice += 3
            else:
                machineChoice = random.randint(0, 2)

        result = checkWin(choice, machineChoice, 2)

        if (result == "Win!"):
            won += 1
        else:
            won -= 2
            if (won < 0):
                won = 0

        print("You chose %s" % choices[choice])
        print("The machine chose %s" % choices[machineChoice])
        print("You %s" % result)
        prevChoice = choice

def expertMode():
    # Expert mode - Menggunakan AI Markov Chain (probabilitas historis) untuk memp
    global probabilitiesRPS
    choices = ["Rock", "Paper", "Scissors"]
    choi = ['r', 'p', 's']
    continuePlaying = True
    prevChoice = ""
    choice = 3
    probRock = 0
    probPaper = 0
    probScissors = 0

    try:
        choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))

```

```

except ValueError:
    print("you must enter an integer \n")

if((choice > 2 or choice < 0)):
    print ("You must enter an integer less than three and greater than or equal")
    while((choice > 2 or choice < 0)):
        print ("You must enter an integer less than three and greater than or equal")
        try:
            choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))
        except ValueError:
            print("you must enter an integer \n")

    # Pilihan acak mesin
    machineChoice = random.randint(0, 2)
    result = checkWin(choice,machineChoice,3)
    print ("You chose %s" % choices[choice])
    print ("The machine chose %s" % choices[machineChoice])
    print("You %s" % result)

prevChoice = choice

while(continuePlaying):
    choice = 3
    try:
        choice = int(input("0: Rock, 1: Paper, 2: Scissors, 5: exit \n"))
    except ValueError:
        print("you must enter an integer \n")

    if((choice > 2 or choice < 0) and choice != 5):
        print ("You must enter an integer less than three and greater than or equal")
        while((choice > 2 or choice < 0) and choice != 5):
            print ("You must enter an integer less than three and greater than or equal")
            try:
                choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))
            except ValueError:
                print("you must enter an integer \n")
    if (choice == 5):
        print("Thanks for Playing!\n")
        continuePlaying = False
    else:
        transMatrix = buildTransitionProbabilities(prevChoice,choice,result)
        machineChoice = random.randint(1, 100)
        probabilitiesRPS[0] = transMatrix[prevChoice][0]
        probabilitiesRPS[1] = transMatrix[prevChoice][1]
        probabilitiesRPS[2] = transMatrix[prevChoice][2]
        rangeR = probabilitiesRPS[0] * 100
        rangeP = probabilitiesRPS[1] * 100 + rangeR
        if (machineChoice <= rangeR):
            machineChoice = 1
        elif (machineChoice <= rangeP):
            machineChoice = 2
        else:
            machineChoice = 0

    result = checkWin(choice,machineChoice,3)
    prevChoice = choice
    print ("You chose %s" % choices[choice])
    print ("The machine chose %s" % choices[machineChoice])
    print("You %s" % result)

```



```

#print("Your winning transition matrix is:\nr: %s\np: %s\ns: %s\n" % (tMatrix[
#print("Your losing transition matrix is:\nr: %s\np: %s\ns: %s\n" % (tMatrixL[
#print("Your tying transition matrix is:\nr: %s\np: %s\ns: %s\n" % (tMatrixT[0

def buildTransitionProbabilities(pC,c,winloss):
    global buildTMatrix
    global buildTMatrixL
    global buildTMatrixT
    choi = ['r','p','s']

    if winloss == "Win!":
        for i, x in buildTMatrix.items():
            if ('%s%s' % (choi[pC],choi[c])) == i):
                buildTMatrix['%s%s' % (choi[pC], choi[c])] += 1
    elif winloss == "Tied!":
        for i, x in buildTMatrixT.items():
            if ('%s%s' % (choi[pC],choi[c])) == i):
                buildTMatrixT['%s%s' % (choi[pC], choi[c])] += 1
    else:
        for i, x in buildTMatrixL.items():
            if ('%s%s' % (choi[pC],choi[c])) == i):
                buildTMatrixL['%s%s' % (choi[pC], choi[c])] += 1

    return buildTransitionMatrix(winloss)

def buildTransitionMatrix(winlosstwo):
    global tMatrix
    global tMatrixL
    global tMatrixT

    if winlosstwo == "Win!":
        rock = buildTMatrix['rr'] + buildTMatrix['rs'] +buildTMatrix['rp']
        paper = buildTMatrix['pr'] + buildTMatrix['ps'] +buildTMatrix['pp']
        scissors = buildTMatrix['sr'] + buildTMatrix['ss'] +buildTMatrix['sp']
        choi = ['r','p','s']
        for row_index, row in enumerate(tMatrix):
            for col_index, item in enumerate(row):
                a = int(buildTMatrix['%s%s' % (choi[row_index],choi[col_index])])
                if (row_index == 0):
                    c = a/rock
                elif (row_index == 1):
                    c = a/paper
                else:
                    c = a/scissors
                row[col_index] = float(c)
        return (tMatrix)
    elif winlosstwo == "Tied!":
        rock = buildTMatrixT['rr'] + buildTMatrixT['rs'] +buildTMatrixT['rp']
        paper = buildTMatrixT['pr'] + buildTMatrixT['ps'] +buildTMatrixT['pp']
        scissors = buildTMatrixT['sr'] + buildTMatrixT['ss'] +buildTMatrixT['sp']
        choi = ['r','p','s']
        for row_index, row in enumerate(tMatrixT):
            for col_index, item in enumerate(row):
                a = int(buildTMatrixT['%s%s' % (choi[row_index],choi[col_index])])
                if (row_index == 0):
                    c = a/rock
                elif (row_index == 1):
                    c = a/paper
                else:
                    c = a/scissors

```

```

        row[col_index] = float(c)
    return (tMatrixT)

else:
    rock = buildTMatrixL['rr'] + buildTMatrixL['rs'] + buildTMatrixL['rp']
    paper = buildTMatrixL['pr'] + buildTMatrixL['ps'] + buildTMatrixL['pp']
    scissors = buildTMatrixL['sr'] + buildTMatrixL['ss'] + buildTMatrixL['sp']
    choi = ['r','p','s']
    for row_index, row in enumerate(tMatrixL):
        for col_index, item in enumerate(row):
            a = int(buildTMatrixL['%s%s' % (choi[row_index],choi[col_index])])
            if (row_index == 0):
                c = a/rock
            elif (row_index == 1):
                c = a/paper
            else:
                c = a/scissors
            row[col_index] = float(c)
    return (tMatrixL)
def superHard():
    # Super Hard Mode - Bot menggunakan kombinasi Markov dan manipulasi input yang
    choices = ["Rock","Paper","Scissors"]
    continuePlaying = True
    continueGame = ""
    result = ""
    print ("I am going to play %s" % choices[random.randint(0, 2)])
    try:
        choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))
    except ValueError:
        print("you must enter an integer \n")

    if((choice > 2 or choice < 0)):
        print ("You must enter an integer less than three and greater than or equal")
        while((choice > 2 or choice < 0)):
            try:
                print ("You must enter an integer less than three and greater than or eq
                choice = int(input("0: Rock, 1: Paper, 2: Scissors \n"))
            except ValueError:
                print("you must enter an integer \n")

    machineChoice = choice - 2
    if (machineChoice < 0):
        machineChoice += 3

    result = checkWin(choice,machineChoice,4)
    print ("You chose %s" % choices[choice])
    print ("The machine chose %s" % choices[machineChoice])
    print("You %s" % result)

    while(continuePlaying):
        print ("I am going to play %s" % choices[random.randint(0, 2)])
        try:
            choice = int(input("0: Rock, 1: Paper, 2: Scissors, 5: exit \n"))
        except ValueError:
            print("you must enter an integer \n")

        if((choice > 2 or choice < 0) and choice != 5):
            print ("You must enter an integer less than three and greater than or equa
            while((choice > 2 or choice < 0) and choice != 5):
                try:

```

```

        print ("You must enter an integer less than three and greater than or
        choice = int(input("0: Rock, 1: Paper, 2: Scissors, 5: exit \n"))
    except ValueError:
        print("you must enter an integer \n")

    if (choice == 5):
        print("Giving up? How sad :(")
        continuePlaying = False
    else:
        machineChoice = choice - 2
        if (machineChoice < 0):
            machineChoice += 3

        result = checkWin(choice,machineChoice,4)
        print ("You chose %s" % choices[choice])
        print ("The machine chose %s" % choices[machineChoice])
        print("You %s" % result)

def bigbang():
    # Big Bang Mode - Mode curang dimana bot selalu mengalahkan pilihan pemain
    global hiddenfound
    global probabilitiesRPS
    choices = ["Rock","Paper","Scissors","Lizard","Spock"]
    choi = ['r','p','sc','l','sp']
    continuePlaying = True
    prevChoice = ""
    choice = 8
    probRock = 0
    probPaper = 0
    probScissors = 0
    probLizard = 0
    probSpock = 0

    try:
        choice = int(input("0: Rock, 1: Paper, 2: Scissors, 3: Lizard, 4: Spock \n"))
    except ValueError:
        print("you must enter an integer \n")

    if((choice > 4 or choice < 0)):
        print ("You must enter an integer less than five and greater than or equal t
        while((choice > 4 or choice < 0)):
            print ("You must enter an integer less than five and greater than or equal
            try:
                choice = int(input("0: Rock, 1: Paper, 2: Scissors, 3: Lizard, 4: Spock
            except ValueError:
                print("you must enter an integer \n")

    machineChoice = random.randint(0, 4)
    result = checkWin(choice,machineChoice,73)
    print ("You chose %s" % choices[choice])
    print ("The machine chose %s" % choices[machineChoice])
    print("You %s" % result)

    prevChoice = choice

    while(continuePlaying):
        choice = 8
        try:
            choice = int(input("0: Rock, 1: Paper, 2: Scissors, 3: Lizard, 4: Spock, 7
        except ValueError:

```

```

print("you must enter an integer \n")

if((choice > 4 or choice < 0) and choice != 7):
    print ("You must enter an integer less than five and greater than or equal
while((choice > 4 or choice < 0) and choice != 7):
    print ("You must enter an integer less than five and greater than or equ
try:
    choice = int(input("0: Rock, 1: Paper, 2: Scissors, 3: Lizard, 4: Spoc
except ValueError:
    print("you must enter an integer \n")
if (choice == 7):
    hiddenfound = True
    print ("\033[1;36m\nThere's just no pleasing you, is there, Leonard?" + X)
    continuePlaying = False
else:
    transMatrix = buildTransitionProbabilitiesrpscfsp(prevChoice,choice,result
    machineChoice = random.randint(1, 100)
    probabilitiesrpscfsp[0] = transMatrix[prevChoice][0]
    probabilitiesrpscfsp[1] = transMatrix[prevChoice][1]
    probabilitiesrpscfsp[2] = transMatrix[prevChoice][2]
    probabilitiesrpscfsp[3] = transMatrix[prevChoice][3]
    probabilitiesrpscfsp[4] = transMatrix[prevChoice][4]
    rangeR = probabilitiesrpscfsp[0] * 100
    rangeP = probabilitiesrpscfsp[1] * 100 + rangeR
    rangeSC = probabilitiesrpscfsp[2] * 100 + rangeP
    rangeL = probabilitiesrpscfsp[3] * 100 + rangeSC
    oneOrTwo = random.randint(1,2)
    if (machineChoice <= rangeR):
        if (oneOrTwo == 1):
            machineChoice = 1
        else:
            machineChoice = 4
    elif (machineChoice <= rangeP):
        if (oneOrTwo == 1):
            machineChoice = 2
        else:
            machineChoice = 3
    elif (machineChoice <= rangeSC):
        if (oneOrTwo == 1):
            machineChoice = 4
        else:
            machineChoice = 0
    elif (machineChoice <= rangeL):
        if (oneOrTwo == 1):
            machineChoice = 2
        else:
            machineChoice = 0
    else:
        if (oneOrTwo == 1):
            machineChoice = 1
        else:
            machineChoice = 3
    result = checkWin(choice,machineChoice,73)
    prevChoice = choice
    print ("You chose %s" % choices[choice])
    print ("The machine chose %s" % choices[machineChoice])
    print("You %s" % result)

```

```

#print("Your winning transition matrix is:\nr: %s\np: %s\nsc: %s\nl: %s\nsp: %
#print("Your losing transition matrix is:\nr: %s\np: %s\nsc: %s\nl: %s\nsp: %s

```

```

#print("Your tying transition matrix is:\nr: %s\np: %s\nsc: %s\nl: %s\nsp: %s\n")

def buildTransitionProbabilitiesrpsclsp(pC,c,winloss):
    global buildTMatrixrpsclsp
    global buildTMatrixLrpsclsp
    global buildTMatrixTrpsclsp
    choi = ['r','p','sc','l','sp']

    if winloss == "Win!":
        for i, x in buildTMatrixrpsclsp.items():
            if ('%s%s' % (choi[pC],choi[c]) == i):
                buildTMatrixrpsclsp['%s%s' % (choi[pC], choi[c])] += 1
    elif winloss == "Tied!":
        for i, x in buildTMatrixTrpsclsp.items():
            if ('%s%s' % (choi[pC],choi[c]) == i):
                buildTMatrixTrpsclsp['%s%s' % (choi[pC], choi[c])] += 1
    else:
        for i, x in buildTMatrixLrpsclsp.items():
            if ('%s%s' % (choi[pC],choi[c]) == i):
                buildTMatrixLrpsclsp['%s%s' % (choi[pC], choi[c])] += 1

    return buildTransitionMatrixrpsclsp(winloss)

def buildTransitionMatrixrpsclsp(winlosstwo):
    global tMatrixrpsclsp
    global tMatrixTrpsclsp
    global tMatrixLrpsclsp

    if winlosstwo == "Win!":
        rock = buildTMatrixrpsclsp['rr'] + buildTMatrixrpsclsp['rsc'] + buildTMatrixrpsclsp['rsc'] + buildTMatrixrpsclsp['rsc']
        paper = buildTMatrixrpsclsp['pr'] + buildTMatrixrpsclsp['psc'] + buildTMatrixrpsclsp['psc'] + buildTMatrixrpsclsp['psc']
        scissors = buildTMatrixrpsclsp['scr'] + buildTMatrixrpsclsp['scsc'] + buildTMatrixrpsclsp['scsc'] + buildTMatrixrpsclsp['scsc']
        lizard = buildTMatrixrpsclsp['lr'] + buildTMatrixrpsclsp['lsc'] + buildTMatrixrpsclsp['lsc'] + buildTMatrixrpsclsp['lsc']
        spock = buildTMatrixrpsclsp['spr'] + buildTMatrixrpsclsp['spsc'] + buildTMatrixrpsclsp['spsc'] + buildTMatrixrpsclsp['spsc']
        choi = ['r','p','sc','l','sp']
        for row_index, row in enumerate(tMatrixrpsclsp):
            for col_index, item in enumerate(row):
                a = int(buildTMatrixrpsclsp['%s%s' % (choi[row_index],choi[col_index])])
                if (row_index == 0):
                    c = a/rock
                elif (row_index == 1):
                    c = a/paper
                elif (row_index == 2):
                    c = a/scissors
                elif (row_index == 3):
                    c = a/lizard
                else:
                    c = a/spock
                row[col_index] = float(c)
            return (tMatrixrpsclsp)
    elif winlosstwo == "Tied!":
        rock = buildTMatrixTrpsclsp['rr'] + buildTMatrixTrpsclsp['rsc'] + buildTMatrixTrpsclsp['rsc'] + buildTMatrixTrpsclsp['rsc']
        paper = buildTMatrixTrpsclsp['pr'] + buildTMatrixTrpsclsp['psc'] + buildTMatrixTrpsclsp['psc'] + buildTMatrixTrpsclsp['psc']
        scissors = buildTMatrixTrpsclsp['scr'] + buildTMatrixTrpsclsp['scsc'] + buildTMatrixTrpsclsp['scsc'] + buildTMatrixTrpsclsp['scsc']
        lizard = buildTMatrixTrpsclsp['lr'] + buildTMatrixTrpsclsp['lsc'] + buildTMatrixTrpsclsp['lsc'] + buildTMatrixTrpsclsp['lsc']
        spock = buildTMatrixTrpsclsp['spr'] + buildTMatrixTrpsclsp['spsc'] + buildTMatrixTrpsclsp['spsc'] + buildTMatrixTrpsclsp['spsc']
        choi = ['r','p','sc','l','sp']
        for row_index, row in enumerate(tMatrixTrpsclsp):
            for col_index, item in enumerate(row):
                a = int(buildTMatrixTrpsclsp['%s%s' % (choi[row_index],choi[col_index])])

```

```

        if (row_index == 0):
            c = a/rock
        elif (row_index == 1):
            c = a/paper
        elif (row_index == 2):
            c = a/scissors
        elif (row_index == 3):
            c = a/lizard
        else:
            c = a/spock
        row[col_index] = float(c)
    return (tMatrixTrpsclsp)
else:
    rock = buildTMatrixLrpsclsp['rr'] + buildTMatrixLrpsclsp['rsc'] + buildT
    paper = buildTMatrixLrpsclsp['pr'] + buildTMatrixLrpsclsp['psc'] + build
    scissors = buildTMatrixLrpsclsp['scr'] + buildTMatrixLrpsclsp['scsc'] +
    lizard = buildTMatrixLrpsclsp['lr'] + buildTMatrixLrpsclsp['lsc'] + buil
    spock = buildTMatrixLrpsclsp['spr'] + buildTMatrixLrpsclsp['spsc'] + bui
    choi = ['r','p','sc','l','sp']
    for row_index, row in enumerate(tMatrixLrpsclsp):
        for col_index, item in enumerate(row):
            a = int(buildTMatrixLrpsclsp['%s%s' % (choi[row_index],choi[col_
            if (row_index == 0):
                c = a/rock
            elif (row_index == 1):
                c = a/paper
            elif (row_index == 2):
                c = a/scissors
            elif (row_index == 3):
                c = a/lizard
            else:
                c = a/spock
            row[col_index] = float(c)
    return (tMatrixLrpsclsp)

def checkWin(user, machine, mode):
    win = False
    tie = False
    if (mode == 73):
        if (user == 0):
            if (machine == 2 or machine == 3):
                win = True
                tie = False
            elif (machine == 1 or machine == 4):
                win = False
                tie = False
            elif (machine == 0):
                tie = True
            else:
                print ("Something wierd happened and machine was: %s" % machine)
        elif (user == 1):
            if (machine == 0 or machine == 4):
                win = True
                tie = False
            elif (machine == 2 or machine == 3):
                win = False
                tie = False
            elif (machine == 1):
                tie = True
            else:

```

```

        print ("Something wierd happened and machine was: %s" % machine)
    elif (user == 2):
        if (machine == 1 or machine == 3):
            win = True
            tie = False
        elif (machine == 0 or machine == 4):
            win = False
            tie = False
        elif (machine == 2):
            tie = True
        else:
            print ("Something wierd happened and machine was: %s" % machine)
    elif (user == 3):
        if (machine == 4 or machine == 1):
            win = True
            tie = False
        elif (machine == 2 or machine == 0):
            win = False
            tie = False
        elif (machine == 3):
            tie = True
        else:
            print ("Something wierd happened and machine was: %s" % machine)
    else:
        if (machine == 2 or machine == 0):
            win = True
            tie = False
        elif (machine == 1 or machine == 3):
            win = False
            tie = False
        elif (machine == 4):
            tie = True
        else:
            print ("Something wierd happened and machine was: %s" % machine)
else:
    if (user == 0):
        if (machine == 2):
            win = True
            tie = False
        elif (machine == 1):
            win = False
            tie = False
        elif (machine == 0):
            tie = True
        else:
            print ("Something wierd happened and machine was: %s" % machine)
    elif (user == 1):
        if (machine == 0):
            win = True
            tie = False
        elif (machine == 2):
            win = False
            tie = False
        elif (machine == 1):
            tie = True
        else:
            print ("Something wierd happened and machine was: %s" % machine)
    else:
        if (machine == 1):
            win = True

```

```

        tie = False
    elif (machine == 0):
        win = False
        tie = False
    elif (machine == 2):
        tie = True
    else:
        print ("Something wierd happened and machine was: %s" % machine)

    if (tie == True):
        checkStats(2, mode)
        return "Tied!"
    elif (win):
        checkStats(0, mode)
        return "Win!"
    else:
        checkStats(1, mode)
        return "Lose!"

def main():
    # Main Loop menu mode permainan
    playAgain = True
    notyesorno = True

    while (playAgain):
        notyesorno = True
        playAgain = False
        chosenMode = chooseMode()
        if(chosenMode == 1):
            easyMode()
            print(HighB)
            print ("Your stats:")
            print(X)
            displaystats(winEas,loseEas,tieEas,"Easy Mode")
            displayOtherModes("Easy Mode")
        elif (chosenMode == 2):
            intermediateMode()
            print(HighB)
            print ("Your stats:")
            print(X)
            displaystats(winInt,loseInt,tieInt,"Intermediate Mode")
            displayOtherModes("Intermediate Mode")
        elif (chosenMode == 3):
            expertMode()
            print(HighB)
            print ("Your stats:")
            print(X)
            displaystats(winExp,loseExp,tieExp,"Expert Mode")
            displayOtherModes("Expert Mode")
        elif (chosenMode == 4):
            superHard()
            print(HighB)
            print ("Your stats:")
            print(X)
            displaystats(winHard,loseHard,tieHard,"Super Hard")
            displayOtherModes("Super Hard Mode")
        elif (chosenMode == 73):
            bigbang()
            print(HighB)
            print ("Your stats:")

```



```

        print(X)
        displaystats(winspec,losespec,tiespec,"Big Bang Mode")
        displayOtherModes("Big Bang Mode")
    else:
        print ("I guess we will move on to whether or not ya wanna play again...\n")

    while(notyesorno):
        continueGame = input("Do you wanna play again? Type Yes or No \n")
        if (continueGame.lower() == "yes"):
            print ("Coolio. \n")
            notyesorno = False
            playAgain = True
        elif (continueGame.lower() == "no"):
            print ("Aw that's too bad. :( \n")
            finalstats()
            notyesorno = False
            playAgain = False
        else:
            print ("Nah... That's not an acceptable answer. Please type Yes or No")
            notyesorno = True

def finalstats():
    # Mencetak hasil rekap seluruh statistik saat game berakhir
    global winEas,loseEas,tieEas,winInt,loseInt,tieInt,winHard,loseHard,tieHard,wi
    print(HighB)
    print ("These are your final stats:")
    print (X)
    #Easy Mode
    if(winEas+loseEas+tieEas != 0):
        percentWonE = "{percent:.2%}".format(percent=(winEas / (winEas+loseEas+tieEa
    else:
        percentWonE = "{percent:.2%}".format(percent= 0)
    #Intermediate Mode
    if(winInt+loseInt+tieInt != 0):
        percentWonI = "{percent:.2%}".format(percent=(winInt / (winInt+loseInt+tieIn
    else:
        percentWonI = "{percent:.2%}".format(percent= 0)
    #Expert Mode
    if(winExp+loseExp+tieExp != 0):
        percentWonEx = "{percent:.2%}".format(percent=(winExp / (winExp+loseExp+tieE
    else:
        percentWonEx = "{percent:.2%}".format(percent= 0)
    #Super Hard Mode
    if(winHard+loseHard+tieHard != 0):
        percentWonH = "{percent:.2%}".format(percent=(winHard / (winHard+loseHard+ti
    else:
        percentWonH = "{percent:.2%}".format(percent= 0)
    #Big Bang Mode
    if(winspec+losespec+tiespec != 0):
        percentWonB = "{percent:.2%}".format(percent=(winspec / (winspec+losespec+ti
    else:
        percentWonB = "{percent:.2%}".format(percent= 0)
    print(Bold)
    print ("You have a %s win rate on %s!" % (percentWonE,"Easy Mode"))
    print ("You have a %s win rate on %s!" % (percentWonI,"Intermediate Mode"))
    print ("You have a %s win rate on %s!" % (percentWonEx,"Expert Mode"))
    print ("You have a %s win rate on %s!" % (percentWonH,"Super Hard Mode"))

    if(hiddenfound == False):
        print("YOU NEVER FOUND THE HIDDEN MODE SCRUB!!!")

```

```

else:
    print("You have a %s win rate on %s!" % (percentWonB,"Big Bang Mode"))
    print(X)
def displaystats(wmode,lmode,tmode,mode):
    print ("\nYou won %d times!\n" % int(wmode))
    print ("You lost %d times!\n" % int(lmode))
    print ("You tied %d times!\n" % int(tmode))
    if(wmode+lmode+tmode != 0):
        percentWon = "{percent:.2%}".format(percent=(wmode / (wmode+lmode+tmode)))
    else:
        percentWon = "{percent:.2%}".format(percent= 0)
    print ("You have a %s win rate on %s! \n" % (percentWon,mode))

def displayOtherModes(mode):
    global winEas,loseEas,tieEas,winInt,loseInt,tieInt,winHard,loseHard,tieHard,wi
    modes = ["Easy Mode", "Intermediate Mode","Expert Mode","Super Hard Mode","Big
    print(HighB)
    print ("Your stats in other modes:")
    print(X)
    if (hiddenfound == False):
        for m in modes:
            if (m != mode):
                print(Bold)
                if (m != "Big Bang Mode"):
                    print (m)
                else:
                    print ("Hidden Mode not yet discovered!!!")
                print(X)
            if (m == "Easy Mode"):
                displaystats(winEas,loseEas,tieEas,"Easy Mode")
            if (m == "Intermediate Mode"):
                displaystats(winInt,loseInt,tieInt,"Intermediate Mode")
            if (m == "Expert Mode"):
                displaystats(winExp,loseExp,tieExp,"Expert Mode")
            if (m == "Super Hard Mode"):
                displaystats(winHard,loseHard,tieHard,"Super Hard Mode")
        else:
            for m in modes:
                if (m != mode):
                    print(Bold)
                    print (m)
                    print(X)
                    if (m == "Easy Mode"):
                        displaystats(winEas,loseEas,tieEas,"Easy Mode")
                    if (m == "Intermediate Mode"):
                        displaystats(winInt,loseInt,tieInt,"Intermediate Mode")
                    if (m == "Expert Mode"):
                        displaystats(winExp,loseExp,tieExp,"Expert Mode")
                    if (m == "Super Hard Mode"):
                        displaystats(winHard,loseHard,tieHard,"Super Hard Mode")
                    if (m == "Big Bang Mode"):
                        displaystats(winspec,losespec,tiespec,"Big Bang Mode")

def continueGameCheck(ans):
    if (ans.lower() == "yes"):
        return "Yes"
    elif (ans.lower() == "no"):
        return "No"
    else:
        return "Wrong input"

```

```

def checkStats(wlt,modeChosen):
    global winEas
    global loseEas
    global tieEas
    global winInt
    global loseInt
    global tieInt
    global winHard
    global loseHard
    global tieHard
    global winExp
    global loseExp
    global tieExp
    global winspec
    global losespec
    global tiespec

    if (modeChosen == 1):
        if (wlt == 0):
            winEas += 1
        elif (wlt == 1):
            loseEas += 1
        else:
            tieEas += 1
    elif (modeChosen == 2):
        if (wlt == 0):
            winInt += 1
        elif (wlt == 1):
            loseInt += 1
        else:
            tieInt += 1
    elif (modeChosen == 3):
        if (wlt == 0):
            winExp += 1
        elif (wlt == 1):
            loseExp += 1
        else:
            tieExp += 1
    elif (modeChosen == 4):
        if (wlt == 0):
            winHard += 1
        elif (wlt == 1):
            loseHard += 1
        else:
            tieHard += 1
    else:
        if (wlt == 0):
            winspec += 1
        elif (wlt == 1):
            losespec += 1
        else:
            tiespec += 1

main()

```

Welcome to Rock Paper Scissors the Ultimate Version! There are four modes: Beginner, Intermediate, Expert, and Super Hard. Beginner is random, Intermediate uses AI, Expert uses Machine Learning, and Super Hard is... well... super hard. To play, type in the number corresponding to the command unless it asks for a yes or no answer. It's pretty self explanatory tbh so you should be fine.

Oh, and if you try to cheat off my code I WILL FIND YOU. Have fun!

P.S. There is a hidden mode that is pretty easy to figure out if you look at the code but thats no fun so try using your ingenuity to figure it out. Here's a hint : Bazinga!!!

You chose Rock
The machine chose Rock
You Tied!
You chose Paper
The machine chose Rock
You Win!
You chose Scissors
The machine chose Rock
You Lose!
Thanks for Playing!

Your stats:

You won 1 times!

You lost 1 times!

You tied 1 times!

You have a 33.33% win rate on Easy Mode!

Your stats in other modes:

Intermediate Mode

You won 0 times!

You lost 0 times!

You tied 0 times!

You have a 0.00% win rate on Intermediate Mode!

Expert Mode

You won 0 times!

You lost 0 times!

You tied 0 times!

You have a 0.00% win rate on Expert Mode!

Super Hard Mode

You won 0 times!

You lost 0 times!

You tied 0 times!

You have a 0.00% win rate on Super Hard Mode!

Hidden Mode not yet discovered!!!

Aw that's too bad. :(

These are your final stats:

You have a 33.33% win rate on Easy Mode!

You have a 0.00% win rate on Intermediate Mode!

You have a 0.00% win rate on Expert Mode!

You have a 0.00% win rate on Super Hard Mode!

YOU NEVER FOUND THE HIDDEN MODE SCRUB!!!